Lambdas

```
Runnable r1 = () -> System.out.println("My Runnable");
```

Das funktionale Interface **Runnable** benötigt die Implementierung einer Prozedur **void run()**

Die Implementierung wird hier durch ein **Lambda** spezifiziert.

Funktionale Interfaces können Ziel einer Zuweisung eines Lambda-Ausdrucks oder einer Methoden-Referenz sein:

```
public class HelloWorld {
   public static void myPrint() {
      System.out.println("My Runnable");
   }
}
Runnable r1 = HelloWorld::myPrint;
```

Lambdas

Lambda Expressions syntax: **argument -> body**

```
() -> {}                        // No parameters; void result

() -> 42                        // No parameters, expression body
() -> null                      // No parameters, expression body
() -> { return 42; }            // No parameters, block body with return
() -> { System.gc(); }          // No parameters, void block body
```

```
// Complex block body with multiple returns
() -> {
  if (true) return 10;
  else {
    int result = 15;
    for (int i = 1; i < 10; i++)
      result *= i;
    return result;
  }
}
```

Lambdas

Lambda Expressions syntax: **argument -> body**

```
(int x) -> { return x+1; } // Single declared-type argument
(int x) -> x+1             // body paranthesis and return optional
(x) -> x+1                 // Single inferred-type argument
x -> x+1                   // Parenthesis optional for single inferred-
                           // type case

(String s) -> s.length()     // Single declared-type argument
(Thread t) -> { t.start(); } // Single declared-type argument
s -> s.length()              // Single inferred-type argument
t -> { t.start(); }          // Single inferred-type argument

(int x, int y) -> x+y      // Multiple declared-type parameters
(x,y) -> x+y               // Multiple inferred-type parameters

(x, final y) -> x+y        // Illegal: modifier not allowed in lambdas
(x, int y) -> x+y          // Illegal: can't mix inferred and declared
                           // types
```

Streams

From
https://docs.oracle.com/en/java/javase/12/docs/api/java.base/java/util/stream/Stream.html

A **stream** can be seen a wrapper around a data source, allowing to operate on the elements of that data source.

A **stream** is not a data structure that stores elements; instead, it conveys elements from a source through a pipeline of computational operations.

Operations are composed into a **stream pipeline**.
A stream pipeline consists of a **source** (which might be an array, a collection, a generator function, an I/O channel, etc), zero or more **intermediate operations** (which transform a stream into another stream, such as filter(Predicate)), and a **terminal operation** (which produces a result or side-effect, such as count() or forEach(Consumer)).

Streams

…

Intermediate operations return a new stream. They are always lazy; executing an intermediate operation such as filter() does not actually perform any filtering, but instead creates a new stream that, when traversed, contains the elements of the initial stream that match the given predicate. **Traversal of the pipeline source does not begin until the terminal operation of the pipeline is executed.**

Terminal operations, such as Stream.forEach or IntStream.sum, may traverse the stream to produce a result or a side-effect.

After the terminal operation is performed, the stream pipeline is considered **consumed**, and can no longer be used; if you need to traverse the same data source again, you must return to the data source to get a new stream.

Streams und Lambdas

```
private static boolean isPrime(int number) {
   if(number < 2) return false;
   for(int i=2; i<number; i++){
      if(number % i == 0)
          return false;
   }
   return true;
}

private static boolean isPrime(int number) {
   IntPredicate isDivisible = value -> number % value == 0;
   // Functional interface and can therefore be used as the
   // assignment target for a lambda expression
   // requires: Boolean test(T value) - Evaluates predicate on the argument.

   return number > 1 &&
   IntStream.range(2, number) // stream of integers from 2 to number
      .noneMatch(isDivisible);
      // Terminal: Returns true if no element of this stream match the
      // provided predicate
      // Boolean noneMatch(IntPredicate predicate)
      // Boolean noneMatch(Predicate<? super T> predicate) for Stream<T>
}
```

Streams und Lambdas

```
public static int sumWithCondition(
   List<Integer> numbers, Predicate<Integer> predicate) {
      return
         numbers.stream() // a stream from a collection
                .filter(predicate)    // intermediate
                .mapToInt(i -> i)     // intermediate,
                                      // requires applyAsInt(T value)
                                      // of Interface ToIntFunction<T>
                .sum(); // terminal defined for IntStream
                        // needs conversion first via mapToInt
}

List<Integer> numbers = Arrays.asList(1, 2, 3,4,5,6,7);

//sum of all numbers
sumWithCondition(numbers, n -> true)

//sum of all even numbers
sumWithCondition(numbers, i -> i%2==0)

//sum of all numbers greater than 5
sumWithCondition(numbers, i -> i>5)
```

Streams und Lambdas

Stream.forEach: wie Iteratoren, auf den ersten Blick, aber verwendet "Internal Iteration", d.h., die JDK bestimmt, wie die Collection iteriert wird, z.B. ob sequentiell oder parallel.

Beispiel unter Verwendung eines Objekts **roster**, dessen Typ das **Interface Collection** mit Element-Typ **Person** implementiert.

```
for (Person p : roster) {
    if (p.getGender() == Person.Sex.MALE) {
        System.out.println(p.getName());
    }
}
```

Nun mit Stream.forEach:

```
roster
 .stream()                                  // a stream from a collection
 .filter(p -> p.getGender() == Person.Sex.MALE)  // intermediate
 .forEach(p -> System.out.println(p.getName()));  // terminal
```

Streams und Lambdas

Stream.reduce: ein generelle **reduction** Operation.

Beispiel unter Verwendung des Objekts **roster**.

```
Integer totalAge = roster
    .stream()
    .mapToInt(Person::getAge) // method from Person to map to int
    .sum();
```

Nun mit Stream.reduce:

```
Integer totalAgeReduce = roster
   .stream()
   .map(Person::getAge)
   .reduce(
       0,
       (a, b) -> a + b);
```

Reduce: **Parameter 1** spezifiziert den initialen Wert, mit dem die Reduction beginnt, sowie den Default-Rückgabewert.
Parameter 2 spezifiziert die Accumulator-Funktion (Lambda) mit 2 Parametern - das bisherige partielle Resultat und das nächste Element, das in der Reduction verwendet wird.

Streams und Lambdas

```
public class mapToIntExample {
   public static void main(String[] args) {

     String[] s = {"one", "two", "three", "four"};
     Stream<String> stringStream = Stream.of(s);

     IntStream intStream =
                        stringStream.mapToInt(e -> e.length());

     IntSummaryStatistics stats =
         intStream
            // Stream<T> peek(Consumer<? super T> action))
            .peek(System.out::println) // intermediate
            .summaryStatistics();      // terminal

     System.out.println(stats);
   }
}
```

```
Ausgabe:
3 3 5 4
IntSummaryStatistics{count=4, sum=15, min=3, average=3.750000, max=5}
```

Streams und Lambdas

```
// Conventional Java
private static int findSquareOfMaxOddIn4to10(List<Integer> numbers)
{
   int max = 0;
   for (int i : numbers) {
      if (i % 2 != 0 && i > 3 && i < 11 && i > max) {
         max = i;
      }
   }
   return max * max;
}
```

Streams und Lambdas

```
public static int findSquareOfMaxOddIn4to10(List<Integer> numbers)
{
   return numbers.stream().filter(NumberTest::isOdd)
      //Predicate is functional interface and we are using
      //lambdas to initialize it rather than anonymous inner
      //classes
               .filter(NumberTest::isGreaterThan3)
               .filter(NumberTest::isLessThan11)
               .max(Comparator.naturalOrder())
               .map(i -> i * i)
               .get();
}

public static boolean isOdd(int i) {
   return i % 2 != 0;
}

public static boolean isGreaterThan3(int i){
   return i > 3;
}

public static boolean isLessThan11(int i){
   return i < 11;
}
```

```
public interface Comparable<T> {
   public int compareTo(T o);
}
```

max yields an object of type Optional<T>
map applies a mapping of the T value in the object,
get yields the value of type T